DATA SCIENCE & ANALYSIS

# The Ultimate NumPy Handbook

Master Numerical Computing in Python

Author

**RahulSinghShah**

NAVIGATE YOUR LEARNING

# Table of Contents


01 **Datatypes and Attributes**

Introduction to ndarrays, the backbone of Machine Learning

02 **Viewing Arrays & Matrices**

Indexing, slicing, and navigating multi-dimensional data

03 **Manipulating Arrays & Broadcasting**

Arithemtic operations, aggregations, and NumPy's broadcasting magic

04 **Reshaping and Transposing**

Changing data dimensions for model compatibility

05 **Dot Product vs Element-wise**

The core engine behind Neural Networks

06 **Sorting Arrays**

Organizing multi-dimensional data effectively

07 **Practical NumPy**

Turning images into numbers

# 1. Datatypes and Attributes

NumPy (Numerical Python) is the foundational library for Data Science and Machine Learning.

## Why is NumPy so Fast?

Native Python lists are slow because they store pointers to scattered objects in memory. NumPy arrays are **written in C** and store data in a **contiguous block of memory**. This allows your CPU to perform *Vectorization* (applying operations to whole blocks of data at once without a `for` loop).

NumPy's main data type is the **`ndarray`** (N-Dimensional Array).

```
import numpy as np

# Creating arrays
a1 = np.array([1, 2, 3])                    # 1D array (Vector)
a2 = np.array([[1, 2.0, 3.3],               # 2D array (Matrix)
               [4, 5, 6.5]])

print(type(a1))
# Output: <class 'numpy.ndarray'>
```

## Essential Array Attributes

To understand any dataset loaded into a NumPy array, you use its attributes.

| Attribute | What it tells you | Example `a2` output |
| --- | --- | --- |
| `shape` | Dimensions of the array | `(2, 3)` (2 rows, 3 cols) |
| `ndim` | Number of dimensions | `2` |
| `dtype` | Data type of elements | `float64` |
| `size` | Total number of elements | `6` |

```
print(a2.shape)  # Very important for ML to know data shape!
print(a2.ndim)   # 2
```

💡 **Key Concept:** A DataFrame in Pandas is actually built directly on top of 2D NumPy arrays!

# 2. Viewing Arrays & Matrices

Arrays can have multiple dimensions. Navigating them is called **indexing and slicing**.

## The Dimensions

1. **1D Array:** A vector. `[1, 2, 3]`
2. **2D Array:** A matrix (rows and columns).
3. **3D Array:** A tensor (like an image with height, width, and color channels).

```
# Create a 3D array (2 matrices, each 2×3)
a3 = np.array([[[1, 2, 3],
                [4, 5, 6]],
               [[7, 8, 9],
                [10, 11, 12]]])
print(a3.shape) # (2, 2, 3)
```

## Slicing Syntax

The syntax is `array[row_slice, col_slice]`.

```
# Get the first element of a1
print(a1[0]) # 1

# In a 2D matrix (a2), get the first row
print(a2[0]) # [1.  2.  3.3]

# Get the element at 0th row, 2nd column
print(a2[0, 2]) # 3.3

# Slice all rows, but only get the first 2 columns
print(a2[:, :2])
# [[1. 2. ]
#  [4. 5. ]]
```

# 3. Manipulating Arrays & Broadcasting

You can perform arithmetic operations on arrays directly — without needing `for` loops.

```
a1 = np.array([1, 2, 3])
ones = np.ones(3)   # [1. 1. 1.]

# Addition
print(a1 + ones)    # [2. 3. 4.]

# Multiplication
print(a1 * 5)       # [5. 10. 15.]
```

## The Magic of Broadcasting

Broadcasting is how NumPy handles operations on arrays of **different shapes**. The smaller array is "broadcast" across the larger array so that they have compatible shapes.

```
a1 = np.array([1, 2, 3])          # Shape: (3,)
a2 = np.array([[1, 2, 3],         # Shape: (2, 3)
               [4, 5, 6]])

# NumPy broadcasts a1 to match a2's shape!
print(a1 * a2)
# [[ 1  4  9]
#  [ 4 10 18]]
```

⚠️ **Rule of Broadcasting:** Two dimensions are compatible if they are **equal**, or if **one of them is 1**.

## Aggregation Functions

Always use NumPy's built in functions instead of Python's. They are exponentially faster.

```
massive_array = np.random.random(100000)

np.sum(massive_array)   # Fast C-level execution!
np.mean(massive_array)  # Average
np.max(massive_array)   # Highest value
np.var(massive_array)   # Variance: measure of data spread
```

# 4. Reshaping and Transposing

In Machine Learning models, you will constantly face errors like: *"Input shape is incompatible"*. Reshaping and transposing are your solutions.

### `.reshape()`

Changes the shape of an array without changing its data.

```
a2 = np.array([[1, 2, 3],
               [4, 5, 6]]) # Shape: (2, 3)

# Reshape into 3 rows, 2 cols
a2_reshaped = a2.reshape(3, 2)
# [[1, 2],
#  [3, 4],
#  [5, 6]]
```

> **Requirements:** The total size must remain the same! ( `2×3 = 6` elements, `3×2 = 6` elements).

## Transpose ( `.T` )

Transposing means swapping the rows and columns. It literally "flips" the matrix over its diagonal.

```
print(a2.T)
# [[1, 4],
#  [2, 5],
#  [3, 6]]
print(a2.T.shape) # (3, 2)
```

# 5. Dot Product vs Element-wise

This is the most critical math concept for understanding Neural Networks.

## Element-wise Multiplication ( `*` )

Multiplies matching positions. Shapes must match (or broadcast).

```
mat1 = np.array([[1, 2], [3, 4]])
mat2 = np.array([[1, 2], [3, 4]])

print(mat1 * mat2)
# [[1  4]     # (1*1, 2*2)
#  [9 16]]    # (3*3, 4*4)
```

## Dot Product ( `np.dot` or `@` )

Matrix multiplication. The **columns of the first matrix** must equal the **rows of the second matrix**. Example: `(3×2)` matrix dot `(2×3)` matrix = `(3×3)` output.

```
mat1 = np.array([[1, 2],
                 [3, 4],
                 [5, 6]])  # Shape: 3×2

mat2 = np.array([[7, 8],
                 [9, 10]]) # Shape: 2×2

# Multiply!
result = np.dot(mat1, mat2)  # Shape will be 3×2
```

*In Machine Learning, `dot product` is used to multiply input features by neural network weights!*

# 6. Sorting Arrays

Organize your numerical data quickly.

```
random_array = np.array([[3, 5, 1],
                         [8, 2, 6]])

# Sort values along each row (axis 1)
np.sort(random_array)
# [[1, 3, 5],
#  [2, 6, 8]]

# Sort along columns (axis 0)
np.sort(random_array, axis=0)
# [[3, 2, 1],
#  [8, 5, 6]]
```

## `np.argsort()`

Instead of returning the sorted numbers, it returns the **index values** of the sorted numbers. Very useful in ML when you need to know *which item* won the prediction, not just the raw score.

```
a = np.array([10, 5, 1])
print(np.argsort(a)) # [2, 1, 0] (Index 2 is the smallest)
```

# 7. Practical NumPy

You've learned all these matrices — why do they matter? Because **everything in the computer is a number**. Images, sounds, and text are just multi-dimensional arrays.

## Turning an Image into Numbers

Computers view images as a 3D NumPy array: `(Height, Width, Color Channels RGB)`.

```
# To read an image into a NumPy array
from matplotlib.image import imread

image_matrix = imread("car.png")
print(type(image_matrix))   # <class 'numpy.ndarray'>
print(image_matrix.shape)   # e.g., (400, 600, 3) → 400px tal

# Every pixel is a number between 0 and 1.
```

If you can convert real-world items (images, audio, text) into NumPy arrays, a Machine Learning algorithm can learn from them!

# 🎉 Complete!

You've mastered the fundamentals of **NumPy**.

---

NUMPY HANDBOOK

**by RahulSinghShah**